Python Tuples Notes

Based on Tea Types Practice Session

1. Creating a Tuple

```
tea_types = ("black", "green", "oolong")
```

- Tuples use `parentheses ()` with comma-separated values
- Like lists, tuples can hold strings, numbers, or mixed types
- Key difference from lists: tuples are `immutable` — they cannot be changed after creation

2. Accessing Items

```
tea_types[0]    # 'black'  — first item (index 0)
tea_types[-1]   # 'oolong' — last item (negative index)
```

- Indexing works the same as lists: `tuple[index]`
- Negative indexing: `[-1]` returns the last item, `[-2]` the second-to-last, and so on

3. Tuples Are Immutable

```
tea_types[0] = "lemon"
# TypeError: 'tuple' object does not support item assignment
```

- Once created, items in a tuple cannot be added, removed, or changed
- This makes tuples ideal for data that should remain constant — like coordinates, config values, or fixed options

💡 *Error encountered: TypeError: 'tuple' object does not support item assignment — tuples are read-only by design*

4. Getting the Length

```
len(tea_types)   # Returns: 3
```

- `len(tuple)` returns the number of items — works the same as with lists and strings

5. Checking Membership

```
if "green" in tea_types:
    print("yes")
# Output: yes
```

- Use `in` to check whether a value exists in the tuple

- The result is True or False — useful in if conditions

💡 *Error encountered: NameError: name 'yes' is not defined — print(yes) looks for a variable. Use quotes: print("yes")*

6. .count()

```
tea_types.count("green")    # Returns: 1
```

- `.count(value)` returns how many times the given value appears in the tuple
- Returns 0 if the value is not found — no error

7. Checking the Type

```
type(tea_types)
# Returns: <class 'tuple'>
```

- `type(variable)` tells you what data type a variable is
- Helpful for debugging — especially when unsure if something is a `list` or a `tuple`

8. Tuples vs Lists — Key Differences

Feature	List	Tuple
Syntax	`[ ]`	`( )`
Mutable?	`Yes`	`No`
Add items	`.append()`	`Not allowed`
Remove items	`.pop(), del`	`Not allowed`
Indexing	`Yes`	`Yes`
Slicing	`Yes`	`Yes`
in keyword	`Yes`	`Yes`
len()	`Yes`	`Yes`
Use case	`Dynamic data`	`Fixed/constant data`

9. Common Errors Encountered

```
TypeError: 'tuple' object does not support item assignment
```

Tuples are immutable — you cannot change their values after creation. Use a list if you need to modify items.

```
NameError: name 'yes' is not defined
```

Passing a word without quotes makes Python treat it as a variable name. Always use quotes for string values: print("yes").

10. Quick Reference

Operation	Example	Result
Create tuple	`("black", "green", "oolong")`	`New tuple`
Access item	`tea_types[0]`	`'black'`
Negative index	`tea_types[-1]`	`'oolong'`
Length	`len(tea_types)`	`3`
Membership	`"green" in tea_types`	`True`
Count value	`.count("green")`	`1`
Check type	`type(tea_types)`	`<class 'tuple'>`